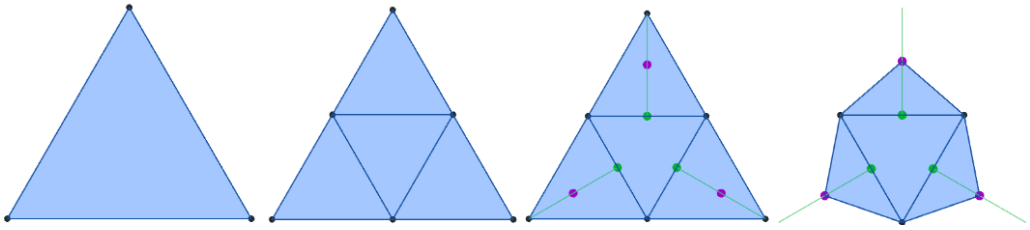
Butterfly algoritmus

1 A számítógépes grafika és 3D-s modellezés területén

A számítógépes grafikában a Butterfly subdivision egy klasszikus, hálók (mesh-ek) simítására és felosztására (subdivision) használt algoritmus, amellyel részletesebb 3D-s felületeket hoznak létre poligonokból.

1.1 Algoritmus lépései

1. Minden élen kiszámoljuk a felezőpontot
2. Minden háromszöget felosztunk 4 kisebb háromszögre a felezőpontok segítségével
3. Az eredeti csúcsok új helyét úgy számoljuk ki, hogy
    a. a csúcs szomszédainak az átlagát számoljuk
    b. vesszük csúcs és az átlag közötti távolság felét
    c. itt fog elhelyezkedni az új csúcs (a felezőpontokat ott hagyjuk, ahol voltak)



4. 1-3 lépéseket ismételjük meg a kisebb háromszögeken

2 Kód részei és magyarázat

2.1 Interaction – interakciók és kirajzolás

```
1  void drawModel() {
2    pushStyle();
3    if (renderMode == 0) {
4      stroke(0, 255, 200, 160); strokeWeight(1); fill(30, 100, 180, 120);
5    } else if (renderMode == 1) {
6      stroke(0, 255, 200, 200); strokeWeight(1); fill(40, 120, 220, 255);
7    } else {
8      noStroke(); fill(40, 120, 220, 255);
9    }
10
11   beginShape(TRIANGLES);
12   for (int[] f : faces) {
13     PVector v1 = vertices.get(f[0]), v2 = vertices.get(f[1]), v3 = vertices.get(f[2]);
14     vertex(v1.x, v1.y, v1.z);
15     vertex(v2.x, v2.y, v2.z);
16     vertex(v3.x, v3.y, v3.z);
17   }
18   endShape();
19   popStyle();
20 }
```

Megadjuk a kirajzoláshoz szükséges stílust majd a faces (lapok) vectoron végigiterálva (12. sor) kirajzoljuk a teljes alakzatot.

További kódok nem igényelnek sok magyarázatot:

- `drawUI()`: Kirajzoljuk az interakciók lehetőségeit és fontosabb információkat
- `mousePressed()`, `mouseDragged()`: Kezeljük az egérmozgatást
- `keyPressed()`: Kezeljük az 1, 2, 3, fel, le, jobbra, balra nyilak lenyomását
- `switchLevel(int lvl)`: Kiszámoljuk, hogy melyik felosztási szintet kell megjeleníteni

2.2 MeshLoader – fájl betöltése

```
1  void loadObjFile(String filename) {
2    vertices = new ArrayList<PVector>();
3    faces = new ArrayList<int[]>();
4
5    String[] lines = loadStrings(filename);
6    if (lines == null) {
7      println("Hiba: Nem található a megadott OBJ fájl: " + filename);
8      return;
9    }
10
11   for (String line : lines) {
12     line = line.trim();
13     if (line.startsWith("v ")) {
14       String[] parts = splitTokens(line, " ");
15       if (parts.length >= 4) {
16         vertices.add(new PVector(float(parts[1]), float(parts[2]), float(parts[3])));
17       }
18     } else if (line.startsWith("f ")) {
19       String[] parts = splitTokens(line, " ");
20       if (parts.length >= 4) {
21         int i0 = parseInt(split(parts[1], '/')[0]) - 1;
22         int i1 = parseInt(split(parts[2], '/')[0]) - 1;
23         int i2 = parseInt(split(parts[3], '/')[0]) - 1;
24         faces.add(new int[]{i0, i1, i2});
25
26         if (parts.length >= 5) {
27           int i3 = parseInt(split(parts[4], '/')[0]) - 1;
28           faces.add(new int[]{i0, i2, i3});
29         }
30       }
31     }
32   }
```

Az egyes OBJ fájlok, amiket használunk kétféle sort tartalmazhat:

- **v** x y z: ’v’ = vertice, x, y és z pedig a pont koordinátáit adja meg
- **f** a b c**:** ’f’ = face, a, b és c pedig azon pontok sorszámát/ indexét, amik kiadják a háromszöget (ha négyszögről beszélünk, akkor az f után 4 szám lesz található

2, 3 sor: létrehozzuk a pontok és oldalak tárolóját

5, 31 sor: beolvassuk és feldolgozzuk az obj fájl tartalmát:

- pontokat vektorokban tárolt koordinátákkal tárolunk
- oldalaknál ketté bontjuk a feldolgozást:
    - ha 3-nál több pont kell egy oldalhoz, azaz nem háromszögekből áll az alakzat, akkor a sokszögeket felbontjuk háromszögekre és azt tároljuk el
    - ki kell vonni egyet a 21,22,23-soroknál pont indexéből, mert az obj 1-től indexel, a processing pedig 0-tól

```
34    if (vertices.size() > 0) {
35      PVector sum = new PVector(0, 0, 0);
36      for (PVector v : vertices) sum.add(v);
37      PVector center = PVector.div(sum, vertices.size());
38
39      float maxDist = 0;
40      for (PVector v : vertices) {
41        v.sub(center);
42        float d = v.mag();
43        if (d > maxDist) maxDist = d;
44      }
45
46      float targetSize = 150.0;
47      if (maxDist > 0) {
48        float scaleFactor = targetSize / maxDist;
49        for (PVector v : vertices) v.mult(scaleFactor);
50      }
51    }
52  }
```

Itt kiszámoljuk a test tömegközéppontját, hogy középre tudjuk helyezni, hogy a megjelenítés szép legyen

2.3 Subdivision – felosztás algoritmusa

- `getOrCreateMidpoint()`: kiszámolja a két pont közötti felezőpontot a térben
- `smoothMesh()`:
    - veszi a pontokat a vertices listából (legyen v az éppen számolt pont)
    - megkeresi a v pont szomszédjait
    - a szomszédok koordinátájából számol egy mediánt
    - a mediánpont és a v pont közötti egyenesen kiszámolja a felezőpontot a térben
    - ez a felezőpont lesz az új v pont, a régit eldobjuk
- `applySubdivisionAndSmoothing()`: kiszámolja az új pontokat (felezőpontokat) és az ezekből alkotott háromszögeket (nagy háromszöget felosztja 4 kisebb háromszögre), majd a legvégén kiszámolja az eredeti nagy háromszög csúcsainak az új helyét a `smoothMesh()` függvénnyel

2.4 Butterfly main – főprogram

```
18 void setup() {
19   size(800, 600, P3D);
20   move = width / 2;
21   surface.setLocation(displayWidth / 2 - width / 2, displayHeight / 2 - height / 2);
22   hint(ENABLE_DEPTH_TEST);
23
24   loadObjFile("cube.obj");
25   levelVertices.add(copyVertices(vertices));
26   levelFaces.add(copyFaces(faces));
27
28   println("Szintek előre számítása...");
29   for (int i = 0; i < 4; i++) {
30     applySubdivisionAndSmoothing();
31     levelVertices.add(copyVertices(vertices));
32     levelFaces.add(copyFaces(faces));
33   }
34   switchLevel(0);
35 }
```

Itt a legfontosabb rész a `setup()` függvényben a 29-33 sorok. Itt kiszámolja előre és eltárolja a 4 szintnek megfelelő pontok koordinátáit és azokkal határolt lapokat. Ezek után a `drawModel()` függvényt alkalmazza a program a `draw()` függvényen belül, ami már csak kiválasztja a szintnek megfelelő pontokat és lapokat, majd megjelníti. Azaz az algoritmust összesen 3x számolja ki az egyes szinteknek megfelelően.

3 Eredmény

